

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº 08

NOMBRE COMPLETO: Loza Regino Miguel Ángel

Nº de Cuenta: 319520011

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 21/10/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Importar un modelo de una nave espacial bien texturizada que tenga dos motores o reactores y añadirle 1 luz puntual y 1 luz de tipo spotlight. Acomodar a la nave flotando

Se busco el modelo de una nave de Star Wars en concreto del Wing-Y Bomber

Se uso el siguiente modelo:



Se uso ya que se encuentra correctamente texturizado y tiene dos moteres muy notorios con los cuales la práctica se puede desarrollar de manera sencilla.

El modelo se descargó de la siguiente página:

https://sketchfab.com/3d-models/star-wars-y-wing-bomber-f260b89a362b4564a5ced176905a8878

Se importo en OpenGL y todo funciono de manera correcta, aunque se tuvo que rotar ya que mantiene los ejes de Blender:



```
//Nave Wing-Y flotando
color = glm::vec3(1.0f, 1.0f, 1.0f);    //blanco para que no tiña la textura
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(0.0f, 10.0f, 0.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::rotate(model, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
model = glm::scale(model, glm::vec3(1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
//Se deja puesta la textura albedo: si Model carga la suya, la usa; si no, queda esta
naveTexture.UseTexture();
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
Nave_M.RenderModel();
```

Definimos la luz spotlight y la pointlight

```
//Luz puntual de la NAVE: naranja, va debajo del fuselaje. La posición de relleno se sobreescribe en cada frame dentro del while
pointLights[0] = PointLight(1.0f, 0.5f, 0.0f,   //color RGB: naranja
    0.0f, 1.0f,                                 //intensidad ambiental, intensidad difusa
    0.0f, 0.0f, 0.0f,                           //posición inicial (se sobreescribe en el while)
    1.0f, 0.045f, 0.0075f);                     //atenuación: constante, lineal, exponencial
pointLightCount++;
```

```
//Spotlight de la NAVE: faro en la nariz, apunta al frente y hacia abajo. Posición y dirección se sobreescriben en cada frame dentro del while
spotLights[0] = SpotLight(1.0f, 1.0f, 0.8f,     //color RGB: luz tipo faro
    0.2f, 1.0f,                                 //intensidad ambiental, intensidad difusa
    0.0f, 0.0f, 0.0f,                           //posición inicial (se sobreescribe en el while)
    0.0f, -1.0f, 0.0f,                          //dirección inicial (se sobreescribe en el while)
    1.0f, 0.014f, 0.0007f,                      //atenuación: constante, lineal, exponencial
    20.0f);                                     //edge: ángulo en grados del centro al borde del cono
spotLightCount++;
```

Ponemos la base para poder unir las luces con jerarquía:

```
//Luces de la NAVE
//baseNave: posición y rotación de la nave (SIN escala). Se usa la MISMA matriz para dibujarla y para sus luces, así siempre van juntas.
//Los valores del translate y los rotate son los de la nave; se cambian aquí.
glm::mat4 baseNave(1.0);
baseNave = glm::translate(baseNave, glm::vec3(0.0f, 10.0f, 0.0f));
baseNave = glm::rotate(baseNave, glm::radians(-90.0f), glm::vec3(1.0f, 0.0f, 0.0f));
baseNave = glm::rotate(baseNave, glm::radians(-90.0f), glm::vec3(0.0f, 0.0f, 1.0f));
```

Calculamos la dirección y orientación de cada luz

```
//Faro (spotlight): los vec4 están en ejes del MODELO (X = hacia la nariz, Z = arriba).
//Posición con w = 1: (8, 0, 0) = 8 unidades al frente del origen del modelo, viaja con la nave.
//Dirección con w = 0: (1, 0, -0.6) = al frente y hacia abajo para que el cono alcance a pegar en el piso.
spotLights[0].SetFlash(glm::vec3(baseNave * glm::vec4(8.0f, 0.0f, 0.0f, 1.0f)),
    glm::normalize(glm::vec3(baseNave * glm::vec4(1.0f, 0.0f, -0.6f, 0.0f))));
```

```
//Luz puntual: (0, 0, -3) = 3 unidades debajo del origen del modelo. Se vuelve a crear cada frame con la posición nueva.
glm::vec3 posLuzPuntualNave = glm::vec3(baseNave * glm::vec4(0.0f, 0.0f, -3.0f, 1.0f));
pointLights[0] = PointLight(1.0f, 0.5f, 0.0f,
    0.0f, 1.0f,
    posLuzPuntualNave.x, posLuzPuntualNave.y, posLuzPuntualNave.z,
    1.0f, 0.045f, 0.0075f);
```

Y con eso tendremos las siguientes luces:



2.- Con una tecla prender y apagar al mismo tiempo las luces añadidas a la nave espacial

Para poder realizar esto tuve que modificar el window.h y window.cpp

Se agrega la función getLuzNave(), que regresa el valor de la variable luzNave. Se usa esta función para saber si las luces de la nave deben estar prendidas o apagadas.

```
//luces de la nave (puntual + spotlight) prendidas / apagadas
bool getLuzNave() { return luzNave; }
```

Se declara la variable privada luzNave de tipo bool. Guarda el estado de las luces de la nave: verdadero si están prendidas y falso si están apagadas.

```
bool luzNave;
```

En el constructor se inicializa luzNave = true, para que las luces de la nave estén prendidas cuando arranca el programa.

```
luzNave = true;         //las luces de la nave empiezan prendidas
```

Cuando se presiona la tecla 1, se invierte el valor de luzNave con el operador 1. Solo se procesa la acción de presionar, no la de mantener ni la de soltar. Así, cada pulsación alterna entre prendido y apagado. Como la luz puntual y el spotlight dependen de esta misma variable, una sola tecla controla las dos luces al mismo tiempo.

```
//1 (Práctica 8): prender / apagar al mismo tiempo la luz puntual y el spotlight de la nave
if (key == GLFW_KEY_1 && action == GLFW_PRESS)
{
    theWindow->luzNave = !theWindow->luzNave;
}
```

Se crean dos arreglos locales, pointLightsActivas y spotLightsActivas, cada uno con su contador, que empiezan en cero. Se reconstruyen en cada ciclo del programa y contienen únicamente las luces que deben enviarse al shader en ese cuadro.

```
//Solo se mandan al shader las luces que estén prendidas.
PointLight pointLightsActivas[MAX_POINT_LIGHTS];
unsigned int pointLightCountActivas = 0;
SpotLight spotLightsActivas[MAX_SPOT_LIGHTS];
unsigned int spotLightCountActivas = 0;
```

Si getLuzNave() es verdadero, se copian al arreglo de activas la luz puntual de la nave (pointLights[0]) y el spotlight o faro (spotLights[0]), y se incrementa el contador de cada tipo. Si es falso, no se copia ninguna y los contadores se quedan en cero.

```
if (mainWindow.getLuzNave()) pointLightsActivas[pointLightCountActivas++] = pointLights[0]; //luz puntual de la nave
if (mainWindow.getLuzNave()) spotLightsActivas[spotLightCountActivas++] = spotLights[0];       //faro (spotlight) de la nave
```

SetPointLights y SetSpotLights envían al shader los arreglos de luces activas junto con su contador. Cuando el contador es cero, el shader no calcula iluminación de ese tipo, así que las luces de la nave quedan apagadas y solo permanece la luz direccional.

```
shaderList[0].SetPointLights(pointLightsActivas, pointLightCountActivas);
shaderList[0].SetSpotLights(spotLightsActivas, spotLightCountActivas);
```

Se muestra el ejemplo:





3.- Añadirle a cada uno de los motores una textura que simule fuego o plasma de color azul saliendo de él y añadir en la posición de cada textura de fuego una luz de tipo puntual de color azul.

Para ello usaremos dos planos en cruzados ubicados en los propulsores los cuales texturizaremos.

Se declaró fuegoTexture.

```
//Textura de plasma azul que simula el fuego de los motores
Texture fuegoTexture;
```

Se creó la malla del fuego como dos planos cuadrados cruzados, uno en XY y otro en XZ, para que sea visible desde cualquier ángulo. Se guarda en meshListModel[1].

```
//Fuego de los motores: dos planos cruzados (uno en XY y otro en XZ) para que se vea desde cualquier ángulo.
unsigned int fuegoIndices[] = {
    0, 1, 2,
    0, 2, 3,
    4, 5, 6,
    4, 6, 7
};

GLfloat fuegoVertices[] = {
    //  x      y      z         u     v         nx    ny    nz
    0.0f, -0.5f, 0.0f,      0.0f, 1.0f,     0.0f, 0.0f, 1.0f,
    0.0f, 0.5f, 0.0f,       1.0f, 1.0f,     0.0f, 0.0f, 1.0f,
    -1.0f, 0.5f, 0.0f,      1.0f, 0.0f,     0.0f, 0.0f, 1.0f,
    -1.0f, -0.5f, 0.0f,     0.0f, 0.0f,     0.0f, 0.0f, 1.0f,

    0.0f, 0.0f, -0.5f,      0.0f, 1.0f,     0.0f, 1.0f, 0.0f,
    0.0f, 0.0f, 0.5f,       1.0f, 1.0f,     0.0f, 1.0f, 0.0f,
    -1.0f, 0.0f, 0.5f,      1.0f, 0.0f,     0.0f, 1.0f, 0.0f,
    -1.0f, 0.0f, -0.5f,     0.0f, 0.0f,     0.0f, 1.0f, 0.0f
};

MeshModel* fuego = new MeshModel();
fuego->CreateMeshModel(fuegoVertices, fuegoIndices, 64, 12);
meshListModel.push_back(fuego);
```

Se carga fuego_azul.png.

```
//Textura del fuego (Textures/fuego_azul.png).
fuegoTexture = Texture("Textures/fuego_azul.png");
fuegoTexture.LoadTextureA();
```

Se definen las dos luces puntuales azules de los motores, pointLights[1] y pointLights[2]. Tienen color azul (0.2, 0.5, 1.0), intensidad ambiental 0.6 y difusa 1.0. Su atenuación es mayor que la de la luz de la nave para que el azul se quede cerca de los motores.

```
//Luces puntuales azules de los MOTORES: pointLights[1] es la del motor 1 (y positiva) y pointLights[2] la del motor 2 (y negativa).
//Las posiciones de relleno se sobreescriben en cada frame dentro del while
pointLights[1] = PointLight(0.2f, 0.5f, 1.0f,   //color RGB: azul
    0.6f, 1.0f,                                 //intensidad ambiental
    0.0f, 0.0f, 0.0f,                           //posición inicial
    1.0f, 0.09f, 0.032f);                       //atenuación
pointLightCount++;
pointLights[2] = PointLight(0.2f, 0.5f, 1.0f,
    0.6f, 1.0f,
    0.0f, 0.0f, 0.0f,
    1.0f, 0.09f, 0.032f);
pointLightCount++;
```

Se multiplica la matriz baseNave por un vector fijo en coordenadas del modelo, así las luces se mueven y rotan junto con la nave.

```
//Luces azules de los motores.
//Se vuelven a crear cada frame con la posición nueva, igual que la luz puntual de la nave.
glm::vec3 posLuzMotor1 = glm::vec3(baseNave * glm::vec4(-28.0f, 6.41f, -0.58f, 1.0f));
pointLights[1] = PointLight(0.2f, 0.5f, 1.0f,
    0.6f, 1.0f,
    posLuzMotor1.x, posLuzMotor1.y, posLuzMotor1.z,
    1.0f, 0.09f, 0.032f);
glm::vec3 posLuzMotor2 = glm::vec3(baseNave * glm::vec4(-28.0f, -6.41f, -0.58f, 1.0f));
pointLights[2] = PointLight(0.2f, 0.5f, 1.0f,
    0.6f, 1.0f,
    posLuzMotor2.x, posLuzMotor2.y, posLuzMotor2.z,
    1.0f, 0.09f, 0.032f);
```

Las dos luces azules se agregan al arreglo de luces activas que se envía al shader.

```
//Luces azules de los motores: por ahora siempre prendidas (con las teclas 2 y 3 se prenderán cada una por separado)
pointLightsActivas[pointLightCountActivas++] = pointLights[1];  //motor 1
pointLightsActivas[pointLightCountActivas++] = pointLights[2];  //motor 2
shaderList[0].SetPointLights(pointLightsActivas, pointLightCountActivas);
shaderList[0].SetSpotLights(spotLightsActivas, spotLightCountActivas);
```

Se dibujan los fuegos después de la nave. Se activa el *blending* con GL_SRC_ALPHA y GL_ONE_MINUS_SRC_ALPHA para respetar la transparencia de la textura, y se desactiva la escritura en el buffer de profundidad (glDepthMask(GL_FALSE)) para que un plano transparente no tape al otro.

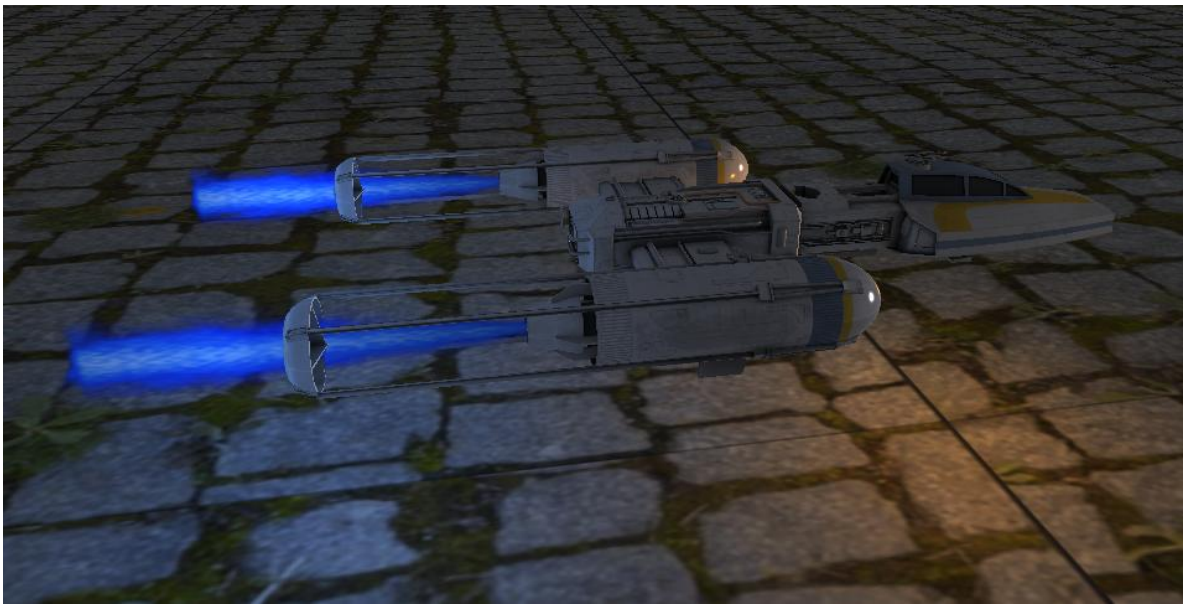
```
//Fuego de los motores: se dibuja DESPUÉS de la nave y con blending (transparencia), igual que el agave.
//glDepthMask(GL_FALSE) evita que las partes transparentes de un plano tapen al plano cruzado.
//Cada fuego usa la matriz de la nave (baseNave), así que viaja con ella. Se coloca en la tobera con el translate (ejes del modelo)
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
glDepthMask(GL_FALSE);
fuegoTexture.UseTexture();
Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);

//Fuego del motor 1 (y positiva): tobera en (-17, 6.41, -0.58)
model = baseNave;
model = glm::translate(model, glm::vec3(-17.0f, 6.41f, -0.58f));
model = glm::scale(model, glm::vec3(15.0f, 4.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshListModel[1]->RenderMeshModel();

//Fuego del motor 2 (y negativa): tobera en (-17, -6.41, -0.58)
model = baseNave;
model = glm::translate(model, glm::vec3(-17.0f, -6.41f, -0.58f));
model = glm::scale(model, glm::vec3(15.0f, 4.0f, 4.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshListModel[1]->RenderMeshModel();

glDepthMask(GL_TRUE);
glDisable(GL_BLEND);
```

Obteniendo lo siguiente:



4.- Con dos teclas independientes añadir que al presionar la tecla se prenda la luz y se vea la textura del fuego de cada motor, de tal forma que se logre ver 4 estados: los dos motores apagados, los dos motores prendidos, motor 1 apagado y motor 2 prendido, motor 1 prendido y motor 2 apagado.

Declaramos getters getMotor1() y getMotor2(), para que main lea el estado de cada motor.

```
//Práctica 8: fuego + luz azul de cada motor, prendidos / apagados con las teclas 2 (motor 1) y 3 (motor 2)
bool getMotor1() { return motor1; }
bool getMotor2() { return motor2; }
```

Variables privadas motor1 y motor2 (bool), que guardan si cada motor está prendido.

```
bool motor1;            //fuego y luz azul del motor 1 prendidos / apagados
bool motor2;            //fuego y luz azul del motor 2 prendidos / apagados
```

Inicializan los dos motores en false (apagados).

```
 motor1 = false;        //los motores empiezan apagados (se prenden con las teclas 2 y 3)
 motor2 = false;
```

```
//2 (Práctica 8): prender / apagar el fuego y la luz azul del motor 1
if (key == GLFW_KEY_2 && action == GLFW_PRESS)
{
    theWindow->motor1 = !theWindow->motor1;
}
//3 (Práctica 8): prender / apagar el fuego y la luz azul del motor 2
if (key == GLFW_KEY_3 && action == GLFW_PRESS)
{
    theWindow->motor2 = !theWindow->motor2;
}
```

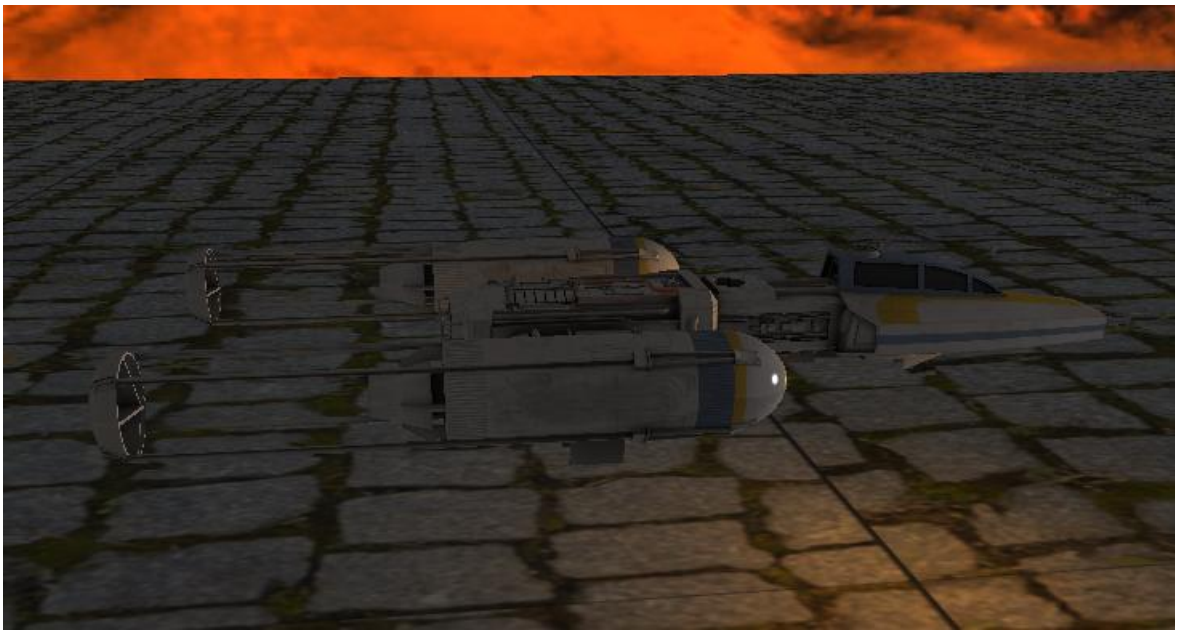
La luz azul de cada motor solo se agrega a las luces activas si su motor está prendido.

```
//Luces azules de los motores: cada una depende de su tecla (2 = motor 1, 3 = motor 2, leídas desde Window)
if (mainWindow.getMotor1()) pointLightsActivas[pointLightCountActivas++] = pointLights[1];  //motor 1
if (mainWindow.getMotor2()) pointLightsActivas[pointLightCountActivas++] = pointLights[2];  //motor 2
shaderList[0].SetPointLights(pointLightsActivas, pointLightCountActivas);
shaderList[0].SetSpotLights(spotLightsActivas, spotLightCountActivas);
```

El fuego de cada motor solo se dibuja si su motor está prendido.

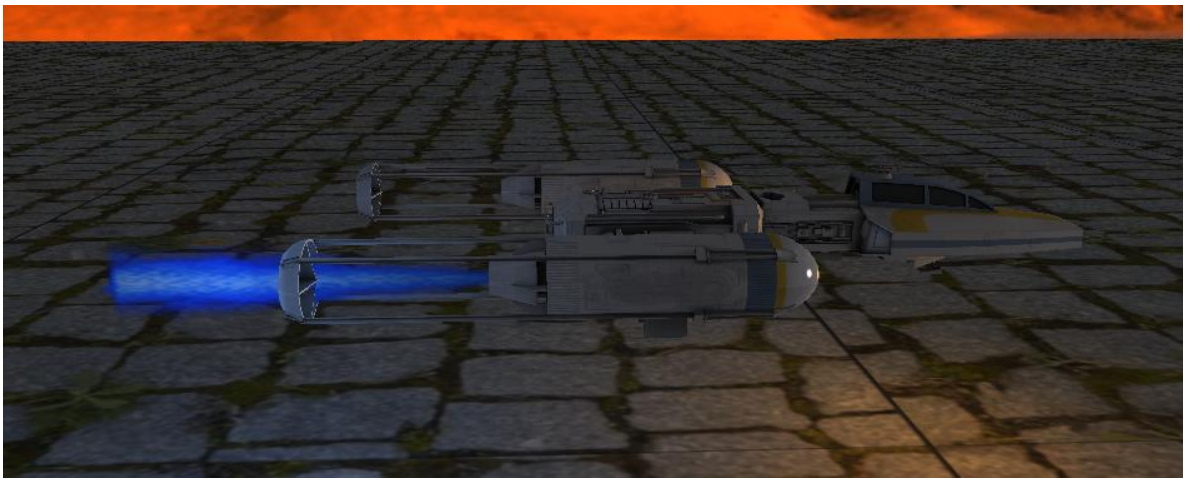
```
if (mainWindow.getMotor1()) meshListModel[1]->RenderMeshModel();    //solo se dibuja si el motor 1 está prendido (tecla 2)
```

```
if (mainWindow.getMotor2()) meshListModel[1]->RenderMeshModel();    //solo se dibuja si el motor 2 está prendido (tecla 3)
```









2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla:

En esta práctica tuve tres problemas principales. El primero fue que al cargar la nave Wing-Y se veía acostada sobre el piso. Pensé que bastaba con rotarla sobre el eje Z, pero al revisar las coordenadas del archivo FBX vi que el modelo viene con

el eje Z hacia arriba, la nariz apuntando a +X y las alas abiertas sobre Y, y la clase Model usa esas coordenadas tal cual. Lo resolví con dos rotaciones de -90° en la matriz base de la nave: la primera sobre el eje X para pararla y la segunda sobre el eje Z para que la nariz apuntara hacia adelante.

El segundo fue que el modelo no trae el fuego de los motores, así que tuve que hacerlo. Creé una malla con dos planos cruzados para que se vea desde cualquier ángulo y una textura PNG de plasma azul con canal alfa. Para que la transparencia se viera bien activé el blending y desactivé la escritura en el buffer de profundidad al dibujar el fuego, porque si no un plano transparente tapaba al otro.

El tercero fue colocar el fuego y las luces azules en cada motor. Para no hacerlo a prueba y error saqué las coordenadas de los anillos traseros de los motores a partir de los vértices del FBX (x = -26.3, y = ±6.41, z = -0.58). Aun así, al principio el fuego salía desde el anillo y se veía separado de la tobera. Lo moví hacia adentro de la jaula (translate en x = -17) y le subí el largo a 15, de modo que ahora nace en la tobera, atraviesa el aro y deja la estela atrás.

3.- Conclusión:

a. Los ejercicios del reporte: Me pareció una práctica interesante porque junté la iluminación con un modelo importado completo y con elementos que yo agregué. Lo que más me ayudó fue entender que las luces y el fuego pueden depender de la misma matriz base de la nave, así todo se mueve y rota junto con ella sin importar dónde la coloque. También entendí mejor cómo se controla el estado de cada luz: como al shader solo se mandan las luces activas, con una tecla independiente por motor se logran los cuatro estados (los dos apagados, los dos prendidos, solo el motor 1 y solo el motor 2). Además, aprendí a usar transparencia con blending para texturas con canal alfa.
b. Comentarios generales: La práctica me gustó porque se nota cómo se juntan los temas de las prácticas anteriores en una sola escena, y la nave con su fuego y sus luces se ve mucho más cercana a algo real. Lo que mejoraría es que el fuego es una textura estática, así que me gustaría animarlo en una práctica futura.

1. Bibliografía en formato APA:

2. de Vries, J. (s. f.). *Blending*. LearnOpenGL. https://learnopengl.com/Advanced-OpenGL/Blending
3. de Vries, J. (s. f.). *Light casters*. LearnOpenGL. https://learnopengl.com/Lighting/Light-casters
4. GLFW. (s. f.). *Input guide*. https://www.glfw.org/docs/latest/input_guide.html

5. LarsH. (s. f.). *Star Wars Y-Wing Bomber* [Modelo 3D]. Sketchfab. https://sketchfab.com/3d-models/star-wars-y-wing-bomber-f260b89a362b4564a5ced176905a8878